

ЛАБОРАТОРНА РОБОТА № 2

з дисципліни «Програмування»

Тема: Робота з умовними операторами, циклами та обчисленням
функцій у мові C

Виконав: студент групи KI-26-1
Волохов Дмитрій
Варіант 3

2026

1. Мета роботи

Метою лабораторної роботи є закріплення практичних навичок програмування мовою C, використання умовних операторів та циклів, організації введення і виведення даних, перевірки коректності вхідних параметрів, а також оцінювання похибки чисельного обчислення.

2. Завдання

Для варіанта 3 розроблено дві консольні програми. Перша програма обчислює значення функції на заданому відрізку з кроком dX та обробляє точку, у якій функція не визначена. Друга програма обчислює $\cos(x)$ за допомогою степеневого ряду з заданою точністю ϵ , порівнює результат із бібліотечним значенням $\cos(x)$ та визначає абсолютну і відносну похибки.

3. Теоретичні відомості

У `task1.c` використано розгалуження `if / else if / else` для вибору відповідної формули залежно від значення x . Для обробки особливої точки $x = 2$ використовується перевірка близькості до нуля та виводиться повідомлення [Not defined]. Також перевіряється, що $Xstart < Xend$ і $dX > 0$.

У `task2.c` використано розклад функції $\cos(x)$ у степеневий ряд: $\cos(x) = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \dots$. Наступний член ряду обчислюється рекурентно, що дозволяє уникнути повторного обчислення факторіалів. Обчислення продовжується, поки модуль поточного члена не стане меншим за ϵ .

4. Алгоритм виконання

Task 1: зчитати $Xstart$, $Xend$ і dX → перевірити коректність → пройти цикл по x → вибрати формулу → вивести результат.

Task 2: зчитати x і ϵ → перевірити $\epsilon > 0$ → ініціалізувати перший член ряду → обчислювати наступні члени до досягнення заданої точності → порівняти суму з $\cos(x)$ → вивести похибки та кількість ітерацій.

5. Програмний код

5.1. Файл task1.c

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    double x_start, x_end, dx;

    printf("Enter Xstart, Xend and dX: ");

    if (scanf("%lf %lf %lf", &x_start, &x_end, &dx) != 3) {
        printf("Error: invalid input.\n");
        return 1;
    }

    if (x_start >= x_end || dx <= 0.0) {
        printf("Error: Xstart must be less than Xend and dX must be positive.\n");
        return 1;
    }

    printf("\n%12s | %18s\n", "X", "Y(X)");
    printf("-----\n");

    for (double x = x_start; x <= x_end + 1e-12; x += dx) {
        double y;

        if (x <= 1.0) {
            y = 2.0 * x + 1.0;
        } else if (x <= 3.0) {
            if (fabs(x - 2.0) < 1e-12) {
                printf("%12.4f | %18s\n", x, "[Not defined]");
                continue;
            }

            y = x / (x - 2.0);
        } else {
            y = sqrt(x);
        }

        printf("%12.4f | %18.6f\n", x, y);
    }

    return 0;
}
```

5.2. Файл task2.c

```
#include <math.h>
#include <stdio.h>

int main(void)
{
    double x, eps;

    printf("Enter X and epsilon: ");

    if (scanf("%lf %lf", &x, &eps) != 2) {
        printf("Error: invalid input.\n");
        return 1;
    }

    if (eps <= 0.0) {
        printf("Error: epsilon must be positive.\n");
        return 1;
    }

    double sum = 1.0;
    double term = 1.0;
    double exact = cos(x);

    int n = 1;
    int iterations = 0;

    const int MAX_ITER = 100000;

    while (fabs(term) >= eps) {
        term *= (-x * x) / ((2.0 * n - 1.0) * (2.0 * n));
        sum += term;

        n++;
        iterations++;

        if (iterations >= MAX_ITER) {
            printf("Warning: maximum iteration limit reached.\n");
            break;
        }
    }

    double abs_error = fabs(sum - exact);

    double rel_error;

    if (fabs(exact) > 1e-15) {
        rel_error = abs_error / fabs(exact);
    } else {
        rel_error = abs_error;
    }

    printf("\nResults for variant 3 (cos(x)):\n");
    printf("X                = %.10f\n", x);
    printf("epsilon            = %.10e\n", eps);
    printf("Series sum         = %.12f\n", sum);
    printf("cos(X)             = %.12f\n", exact);
    printf("Absolute error     = %.12e\n", abs_error);
    printf("Relative error     = %.12e\n", rel_error);
    printf("Iterations         = %d\n", iterations);

    return 0;
}
```

6. Результати тестування

6.1. Перевірка task1.c

Для коректного тесту використано $X_{start} = 0$, $X_{end} = 4$, $dX = 1$. Отримані значення: $y(0) = 1$, $y(1) = 3$, у точці $x = 2$ функція не визначена, $y(3) = 3$, $y(4) = 2$.

х	Результат
0	1.000000
1	3.000000
2	[Not defined]
3	3.000000
4	2.000000

При введенні від'ємного кроку dX програма коректно відхиляє вхідні дані та виводить повідомлення про помилку.

6.2. Перевірка task2.c

Для $x = 1$ виконано три запуски зі зменшенням ϵ . Результати показали, що зі зменшенням допустимої похибки кількість ітерацій збільшується, а абсолютна та відносна похибки зменшуються.

ϵ	Сума ряду	Абсолютна похибка	Відносна похибка	Ітерації
0.01	0.540277777778	2.452809036202e-05	4.539697516672e-05	3
0.0001	0.540302579365	2.734963395402e-07	5.061924344385e-07	4
0.000001	0.540302305792	2.076257253030e-09	3.842761177868e-09	5

Окремо перевірено некоректне значення $\epsilon = -0.01$. Програма завершує виконання з повідомленням «Error: epsilon must be positive.», тобто некоректний параметр обробляється штатно.

7. Скріншоти виконання

```
090di@DESKTOP-1MK0JG3 UCRT64 ~/projects/programming-sem1/lab02
$ ./task1
Enter Xstart, Xend and dX: 0 4 1

      X |          Y(X)
-----|-----
    0.0000 |      1.000000
    1.0000 |      3.000000
    2.0000 |      [Not defined]
    3.0000 |      3.000000
    4.0000 |      2.000000

090di@DESKTOP-1MK0JG3 UCRT64 ~/projects/programming-sem1/lab02
$ |
```

Рисунок 1 — Коректний запуск task1.c для $X_{start} = 0$, $X_{end} = 4$, $dX = 1$.

```
      3.0000 |      3.000000
      4.0000 |      2.000000

090di@DESKTOP-1MK0JG3 UCRT64 ~/projects/programming-sem1/lab02
$ ./task1
Enter Xstart, Xend and dX: 0 4 -1
Error: Xstart must be less than Xend and dX must be positive.

090di@DESKTOP-1MK0JG3 UCRT64 ~/projects/programming-sem1/lab02
$
```

Рисунок 2 — Перевірка task1.c з некоректним значенням $dX = -1$.

```
090di@DESKTOP-1MK0JG3 UCRT64 ~/projects/programming-sem1/lab02
$ ./task1
Enter Xstart, Xend and dX: 4 0 1
Error: Xstart must be less than Xend and dX must be positive.

090di@DESKTOP-1MK0JG3 UCRT64 ~/projects/programming-sem1/lab02
```

Рисунок 3 — Повторна перевірка обробки некоректного кроку dX .

```

090d1@DESKTOP-1MK0JG3 UCRT64 ~/projects/pro
$ ./task2
Enter X and epsilon: 1 0.01

Results for variant 3 (cos(x)):
X                = 1.00000000000
epsilon          = 1.00000000000e-02
Series sum       = 0.540277777778
cos(X)           = 0.540302305868
Absolute error    = 2.452809036202e-05
Relative error    = 4.539697516672e-05
Iterations       = 3

```

Рисунок 4 — task2.c для $\epsilon = 0.01$.

```

090d1@DESKTOP-1MK0JG3 UCRT64 ~/projects/programm
$ ./task2
Enter X and epsilon: 1 0.0001

Results for variant 3 (cos(x)):
X                = 1.00000000000
epsilon          = 1.00000000000e-04
Series sum       = 0.540302579365
cos(X)           = 0.540302305868
Absolute error    = 2.734969395402e-07
Relative error    = 5.061924344385e-07
Iterations       = 4

```

Рисунок 5 — task2.c для $\epsilon = 0.0001$.

```
$ ./task2
Enter X and epsilon: 1 0.000001

Results for variant 3 (cos(x)):
X                = 1.00000000000
epsilon          = 1.00000000000e-06
Series sum       = 0.540302303792
cos(X)           = 0.540302305868
Absolute error    = 2.076252725303e-09
Relative error    = 3.842761177868e-09
Iterations       = 5
```

Рисунок 6 — task2.c для $\epsilon = 0.000001$.

```
090di@DESKTOP-1MK0JG3 UCRT64 ~/projec
$ ./task2
Enter X and epsilon: 1 -0.01
Error: epsilon must be positive.

090di@DESKTOP-1MK0JG3 UCRT64 ~/projec
```

Рисунок 7 — Перевірка task2.c з некоректним $\epsilon = -0.01$.

8. Висновок

У ході лабораторної роботи було розроблено та протестовано дві програми мовою С для варіанта 3. У першій програмі реалізовано обчислення функції на заданому інтервалі з контролем коректності вхідних параметрів та обробкою точки, де функція не визначена. У другій програмі реалізовано обчислення $\cos(x)$ за допомогою степеневого ряду з контролем точності ε . Для трьох значень ε підтверджено, що зі збільшенням точності зменшується похибка результату. Також перевірено обробку некоректних вхідних даних. Отримані результати відповідають очікуваній поведінці програм.